能肯定 str 是回文字符串；第 22 ～ 28 行是算法 2，将字符串 str 反序存入字符串变量 temp 中，然后在第 28 行调用 equals() 比较两个字符串，如果相等则是回文字符串。

5.2.2 StringBuffer 型字符串

StringBuffer 类支持可修改的字符串，用该类创建的字符串对象允许在中间插入字符和子串，或在尾部追加字符和子串。StringBuffer 能自动增长，从而为该类添加操作准备空间，并且通常预先分配比实际需要更多的空间，以允许空间增长。创建 StringBuffer 型字符串变量有如下 3 种方法：

```
StringBuffer strBuf=new StringBuffer();       // 创建初始容量为 16 的字符串变量
StringBuffer strBuf=new StringBuffer(个数);   // 创建初始容量为 " 个数 " 的字符串变量
StringBuffer strBuf=new StringBuffer(字符串); // 创建初始内容为 " 字符串 " 的字符变量
```

例如，利用第 3 种方法创建字符串变量：

```
StringBuffer strBuf=new StringBuffer("Hello Java");
```

StringBuilder 类与 StringBuffer 类类似，所以不介绍了。表 5.3 给出了 StringBuffer 类的常用方法。

表 5.3 java.lang.StringBuffer 类的常用方法

常用方法	功能说明
public int capacity()	返回调用该方法字符串的容量
public void ensureCapacity(int minimumCapacity)	设置字符串对象的容量
public StringBuffer append(StringBuffer sb)	将 sb 追加到字符串的末尾
public StringBuffer insert(int offset,String str)	在字符串的 offset 位置插入字符串 str
public StringBuffer delete(int start,int end)	将字符串中 start 和 end–1 之间的子串删除后返回
public StringBuffer deleteCharAt(int index)	将字符串中 index 位置的字符删除
public StringBuffer reverse()	将字符串反序后返回
public String substring(int start,int end)	返回 start 和 end–1 之间的子串

5.3 正则表达式



正则表达式（regular expression，简写为 regex）是一个强大的字符串（又称字符序列）处理工具。例如，在编写处理字符串的程序或网页时，经常会查找符合某些复杂规则的字符串。正则表达式就是用于描述这些规则的工具。它可以对字符串进行查找、匹配、分割和替换等操作。正则表达式本质也是一个字符串，这个字符串是按照一定的语法和规范被构造出来作为限定条件的“规则字符串”，这个“规则字符串”描述了一种字符串匹配的模式。说某个字符串匹配某个正则表达式，通常是指这个字符串里有一部分或几部分能满足正则表达式给出的条件。利用正则表达式可以达到如下目的：

（1）给定的字符串是否符合正则表达式的过滤条件或称匹配；
（2）可以通过正则表达式，从字符串中获取想要的特定部分。

5.3.1 正则表达式中的元字符

正则表达式是由普通字符（例如字符 a ～ z）和元字符所组成的字符串。所谓元字符就是具有特殊意义的字符。构造正则表达式的方法就是用多种元字符与运算符将小的表达式结合在一起来构建满足要求的正则表达式。表 5.4 给出了正则表达式中常用的元字符。更多的元字符参见 java.util.regex 包中的 Pattern 类。

表 5.4 正则表达式中常用的元字符

元 字 符	功 能 说 明
^	匹配输入字符串的开始位置
$	匹配输入字符串的结尾位置
\b	匹配单词边界，也就是指单词和空格间的位置
\B	匹配非单词边界
.	代表任何一个字符
\d	digit，代表 0 ～ 9 中的任何一个数字：[0 ～ 9]
\D	代表任何一个非数字字符：[^0 ～ 9]
\s	space，代表任一空格类字符：[\t\n\x0B\f\r]
\S	代表任一非空格类字符：[^\s]
\w	word，代表可用于标识符的字符：[a ～ zA ～ Z_0 ～ 9]
\W	代表不能用于标识符的字符：[^\w]
\p{Lower}	小写字母 [a ～ z]
\p{Upper}	大写字母 [A ～ Z]
\p{ASCII}	ASCII 字符
\p{Alpha}	字符 [\p{Lower}\p{Upper}]
\p{Digit}	数字 [0-9]
\p{Alnum}	字母或数字 [\p{Alpha}\p{Digit}]
\p{Punct}	标点符号：! " # $ % & ' ( ) * + , - . / : ; < = > ? @ [\ ] ^ _ ` { | } ～
\p{Graph}	可视字符 [\p{Alnum}\p{Punct}]
\p{Print}	可打印字符 [\p{Graph}\x20]
\p{Blank}	空格或制表符 [\t]
\p{Cntrl}	控制字符 [\x00 ～ \x1F\x7F]
\p{XDigit}	十六进制数字 [0 ～ 9a ～ fA ～ F]
\p{Space}	空字符 [\t\n\x0B\f\r]

说明：在正则表达式中使用元字符时要在其前面加上转义字符“\”，如元字符“\d”在正则表达式中要写成“\\d”，又如若要想使用普通意义的点字符“.”，必须写成“\.”。

例如，元字符 \d 代表 0 ～ 9 中的任意一个数字，所以与正则表达式 \\dabc 匹配的字符串有 "0abc""1abc""2abc"……"9abc"。

对于元字符 \b 是匹配单词的边界，如“er\\b”可以匹配单词 never 中的 er，但不能匹配 verb 中的 er。而对于元字符 \B 则是匹配非单词的边界，如“er\\B”，可以匹配 verb 中的 er，但不能匹配 never 中的 er。

表 5.4 中的 ^ 和 $ 两个符号是正则表达式的边界定位符。例如，^once 表示该模式只匹配以 once 开头的字符串，如该模式与字符串 once upon a time 匹配，但不与 There once was a man from Guangzhou 匹配。$ 符号用来匹配那些以给定模式结尾的字符串，例如，bucket$，这个模式与 Who kept all of this cash in a bucket 匹配，但与 buckets 不匹配。

在正则表达式中可以使用方括号将若干字符括起来表示一个元字符，该元字符代表方括号中的任何一个字符。表 5.5 给出了常用的方括号表达式。

表 5.5　正则表达式的方括号表达式

方括号表达式中	功 能 说 明
[xyz]	代表 x、y、z 中的任何一个字符
[^xyz]	代表除 x、y、z 以外的任何一个字符
[a–d]	代表 a ～ d 中的任何一个字符
[a–zA–Z]	代表 a ～ z 或 A ～ Z 中的任何一个字符
[a–d[m–q]]	代表 a ～ d 或 m ～ q 中的任何一个字符（并集），等于 [a–dm–q]
[a–z&&[def]]	代表 a ～ z 并且是 d、e、f 中的任何一个字符（交集），等于 [def]
[a–f&&[^de]]	代表 a ～ f 而且不是 d 或 e 的任一字符（差集），等于 [a-cf]，即 a、b、c 和 f

说明：当符号 ^ 在方括号表达式中使用时，表示不接受该方括号表达式中的字符集合。

除了方括号表达式外，在正则表达式中还可以使用限定符，表 5.6 给出了正则表达式中常用的限定符。

表 5.6　正则表达式中常用的限定符

限 定 符	功 能 说 明
X?	? 号前面的 X 最多只出现 1 次，? 等价于 {0,1}
X*	* 号前面的 X 出现 0 次或多次，* 等价于 {0,}
X+	+ 号前面的 X 出现 1 次或多次，+ 等价于 {1,}
X{n}	X 刚好出现 n 次
X{n,}	X 至少出现 n 次

续表

限 定 符	功 能 说 明
X{n,m}	X 出现 n ～ m 次
X|Y	在 | 左右两边的子表达式中任选一项，即或选 X 或选 Y

例如，正则表达式 runoo*b，可以匹配 runob、runoob、runooooooob…等字符串，因为 * 号代表它前面的字符（本例中为 o）可以不出现，也可以出现一次或者多次。

注意： 不能将定位符与限定符一起使用。由于在紧靠换行或者单词边界的前面或后面不能有一个以上位置，因此不允许诸如 ^* 之类的表达式。若要匹配一行文本开始处的文本，应在正则表达式的开始处使用 ^ 字符。不要将 ^ 的这种用法与方括号表达式内的用法混淆。若要匹配一行文本结束处的文本，则应在正则表达式的结束处使用 $ 字符。

下面分析两个正则表达式。

（1）正则表达“^[-]?[0-9]+\.?[0-9]+$”解析为以一个可选的负号（[-]?）开头（^）、跟着 1 个或多个的数字（[0-9]+）和一个可有可无的小数点（\.?），再跟 1 个或多个数字（[0-9]+），并且后面没有其他任何字符（$）。由于元字符 + 与 {1,} 等价，都代表 1 个或多个前面的内容，因此上面的正则表达式可以简化为 ^\-?[0-9]{1,}\.?[0-9]{1,}$。

（2）正则表达式“^Chapter|Section[1-9][0-9]{0,1}$”表示要么匹配行首的单词 Chapter，要么匹配行尾的单词 Section 及跟在其后的任何数字。如果输入的字符串是 Chapter22，那么上面的表达式只匹配单词 Chapter。如果输入的字符串是 Section22，那么该表达式匹配 Section22。

除了方括号表达式外，还可以使用圆括号将正则表达式分组，此时圆括号可以将括起来的若干字符合成一个字符。例如：

```
a(bc)*                    // 代表 a 后面跟 0 个或多个 bc
a(bc){1,5}                // 代表 a 后面跟 1 ～ 5 个 bc
```

下面给出几个常用的正则表达式：

IP 地址：((2[0-4]\\d|25[0-5]|[01]?\\d\\d?)\\.){3}(2[0-4]\\d|25[0-5]|[01]?\\d\\d?)

邮箱地址：^\\w+([-+.]\\w+)*@\\w+([-.]\\w+)*\\.\\w+([-.]\\w+)*$

日期：^\\d{4}\\-(0?[1-9]|1[012])\\-(0?[1-9]|[12][0-9]|3[01])$

汉字：[\u4e00-\u9fa5]{0,}

5.3.2 正则表达式的应用

正则表达式既可以直接用于字符串进行模式匹配，也可以利用 Pattern 和 Matcher 两个类使用正则表达式。下面分别举例说明。

【例 5.7】利用正则表达来分割字符串，并输出被分割出的每个子字符串。

```
1 //FileName: App5_7.java
```

```
2 import java.util.Scanner;                          //导入Scanner类
3 public class App5_7{
4   public static void main(String[] args){
5     String str="他说:我很好.我说:那就好  Hello Good moring";
6     String regex="[\\s\\p{Punct}]+";               //定义正则表达式
7     String[] words=str.split(regex);               //分割出匹配的子字符串
8     for(int i=0;i<words.length;i++){
9       int m=i+1;
10      System.out.println("单词 "+m+":"+words[i]);
11    }
12  }
13 }
```

程序运行结果:

```
单词 1:他说
单词 2:我很好
单词 3:我说
单词 4:那就好
单词 5:Hello
单词 6:Good
单词 7:moring
```

该程序的第 6 行定义了正则表达式，第 7 行用字符串变量 str 调用了参数为正则表达式 regex 的方法 split()，用于对字符串进行分割，并将分割的子字符串存入 String 型数组 words 中。第 8 ～ 11 行利用循环语句输出每个子字符串。

使用正则表达式的另一方式是利用 java.util.regex 包中的 Pattern 和 Matcher 两个类使用正则表达式。这两个类协同工作，使用模式类 Pattern 编译正则表达式，然后使用匹配类 Matcher 在输入序列中进行模式匹配。模式匹配就是检索和指定模式匹配的字符序列。

模式类 Pattern 是正则表达式的编译表示形式。Pattern 类的实例是通过调用其静态方法 compile() 创建的。表 5.7 给出了 Pattern 类的常用方法。

表 5.7　java.util.regex.Pattern 类的常用方法

常用方法	功能说明
public static Pattern compile(String regex)	将给定的正则表达式 regex 编译成模式实例返回
public static Pattern compile(String regex, int flags)	将给定的正则表达式 regex 编译成具有给定标志 flags 的模式实例返回，flags 的取值是 Pattern 类中定义的常量
public Matcher matcher(CharSequence input)	将用于匹配模式的字符序列 input（又称输入序列）作为参数创建 Matcher 对象
public String pattern()	返回用于编译此模式的正则表达式的字符串表示形式，其实就是返回 Pattern.complile(String regex) 的 regex 参数
public String toString()	返回此模式的字符串表示形式

续表

常用方法	功能说明
public String[] split(CharSequence input, int limit)	将给定的输入序列 input 按照此模式的匹配进行分割，limit 参数指定应用模式的次数。如果限制 limit>0，那么模式将被应用最多 limit-1 次；如果 limit<0，那么该模式将被应用尽可能多的次数；如果 limit=0，那么模式将被应用尽可能多的次数，并且末尾的空字符串将被丢弃
public String[] split(CharSequence input)	功能同上，相当于 limit=0 的情况
public static String quote(String s)	返回匹配字符串 s 的字符串形式的正则表达式。参数中的元字符或转义字符并没有特殊意义

说明： matcher() 与 split() 方法中的参数 input 是将要用于模式匹配的字符序列，又称为输入序列，它是 CharSequence 接口类型，该接口定义了一组只读字符。String 以及其他类实现了该接口，因此可以向 matcher() 和 split() 方法传递字符串。

正则表达式必须先编译成类型为 Pattern 的模式对象，模式对象是通过该类调用自己的 compile() 方法创建的。一旦创建了 Pattern 对象，就可以使用该对象调用 matcher() 方法创建 Matcher 对象。然后就可以利用 Matcher 对象调用相应的方法来执行各种模式匹配操作。表 5.8 给出的 Matcher 类的常用方法。

表 5.8 java.util.regex.Matcher 类的常用方法

常用方法	功能说明
public boolean matches()	判断字符序列是否与模式完全匹配
public boolean find()	判断输入序列的子序列是否与模式相匹配。可以重复调用这个方法以查找所有匹配的子序列。对方法的每次调用，都是从上一次离开的位置开始
public boolean find(int start)	判断输入序列从 start 位置开始是否有匹配的子序列
public boolean lookingAt()	对前面的字符串进行匹配，只有匹配到的字符串在最前面才会返回 true
public String group()	返回匹配到的子字符串
public int start()	返回当前匹配到的字符串在原目标字符串中的位置
public int end()	返回当前匹配的字符串末尾字符之后下一个字符的索引位置
public String replaceAll(String replacement)	用字符序列 replacement 替换与模式相匹配的序列，将更新后的输入序列作为字符串返回
public String replaceFirst(String replacement)	用字符串 replacement 替换与模式匹配序列中的第一个子序列后返回

【例 5.8】把用户给定字符串按正则表达式进行匹配并输出，然后再对用户输入的 E-mail 地址是否合法进行验证。

```
1  //FileName: App5_8.java
2  import java.util.Scanner;                    //导入 Scanner 类
3  import java.util.regex.Pattern;              //导入 Pattern 类
4  import java.util.regex.Matcher;              //导入 Matcher 类
5  public class App5_8{
6    public static void main(String[] args){
7      Pattern p=Pattern.compile("ab\\.*c");       //编译正则表达式
8      Matcher m=p.matcher("ab..cxyzab...cxxx");  //创建给定输入序列的匹配器
9      while(m.find())                  //判断输入序列中是否有匹配的子字符串
10       System.out.println(m.group()+":"+m.start()+","+m.end());
11     //下面定义邮箱格式正则表达式
12     String regex="^\\w+([-+.]\\w+)*@\\w+([-.]\\w+)*\\.\\w+([-.]\\w+)*$";
13     String yn="y";
14     Scanner scan=new Scanner(System.in);//创建从键盘上读数据的对象 scan
15     do{
16       System.out.print("请输入邮箱地址: ");
17       String input=scan.next();
18       Pattern pa=Pattern.compile(regex);        //编译正则表达式
19       Matcher ma=pa.matcher(input);             //创建给定输入序列的匹配器
20       if(ma.matches())                          //判断是否匹配
21         System.out.println("邮箱地址正确! ");
22       else
23         System.out.println("邮箱地址格式错! ");
24       System.out.print("是否继续输入: ");
25       yn=scan.next();
26     }while(yn.equalsIgnoreCase("y"));
27   }
28 }
```

程序运行结果:

```
ab..c:0,5
ab...c:8,14
请输入邮箱地址: 139@qq.com ↙
邮箱地址正确!
是否继续输入: n ↙
```

该程序的第 7 行是用 Pattern 类直接调用 compile() 方法将正则表达式”ab\\.*c”编译成模式对象 p；第 8 行则是用模式对象 p 调用 matcher() 方法生成匹配器对象 m；第 9、10 两行在 while 循环中分别用 group()、start() 和 end() 方法将匹配的子串和起止位置输出；第 12 行定义了正则表达式用于匹配 E-mail 邮箱地址；第 18 行是将正则表达式编译成模式对象 pa；第 19 行则是利用该对象 pa 调用 Pattern 类的 matcher() 方法创建匹配对象 ma 进行模式匹配；第 20 行的 if 语句是判断匹配是否成功。